Where Is My Doctor?

Technical Architecture & Core Search Algorithm

V1 Technical Specification • September 2026

Purpose: define the V1 technology stack, system architecture, location model, healthcare search pipeline, geographic search strategy, ranking model, and core technical decisions for a local healthcare discovery platform.

1. Technical Principles

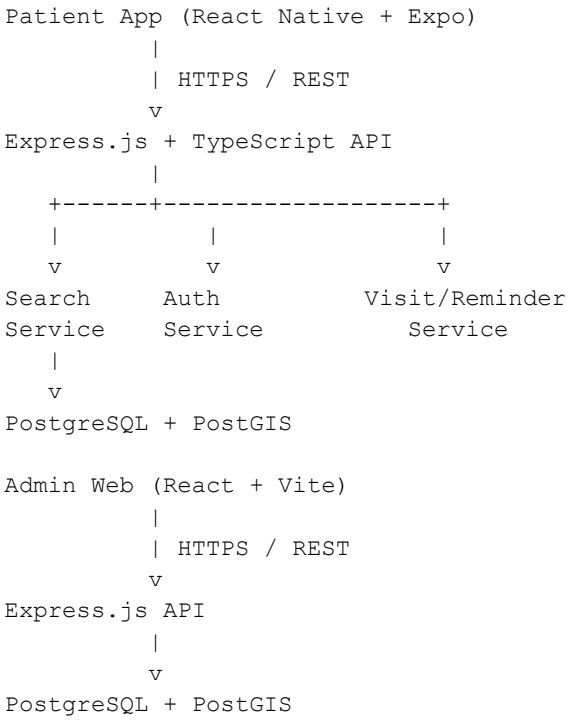
- No paid third-party APIs for the core healthcare discovery/search experience.
- Use device GPS for current coordinates when the user grants location permission.
- Use latitude/longitude and geographic calculations for proximity search.
- Use PostgreSQL as the primary database, with PostGIS for geographic queries.
- Use PostgreSQL full-text search and pg_trgm before introducing a dedicated search engine.
- Keep healthcare search data-driven through a configurable taxonomy rather than hard-coded mappings.
- Keep discovery available without requiring patient login.
- Treat availability and schedule as separate concepts; a scheduled doctor is not automatically guaranteed to be physically present.
- Design the system so paid/external services can be introduced later without changing the core domain model.

2. V1 Technology Stack

Layer	Technology	Purpose
Patient App	React Native + Expo + TypeScript	Android/iOS healthcare discovery app
Admin Web	React + Vite + TypeScript	Healthcare data and platform administration
Backend	Node.js + Express.js + TypeScript	REST API and business logic
Database	PostgreSQL	Primary relational data store
Geospatial	PostGIS	Coordinates, radius filtering and distance calculations
Text Search	PostgreSQL FTS + pg_trgm	Full-text and fuzzy matching
ORM	Drizzle ORM	Type-safe PostgreSQL access
Validation	Zod	Request, response and form validation
Authentication	JWT + refresh tokens	Patient/admin authentication
API Docs	OpenAPI / Swagger	API documentation and testing
Server State	TanStack Query	Caching, synchronization and API state
Web UI	Tailwind CSS + shadcn/ui	Admin interface
Mobile Localization	i18next / react-i18next	English and Bengali localization
Notifications	Expo Notifications	My Visits reminders
Infrastructure	Linux VPS + Nginx	Initial production deployment

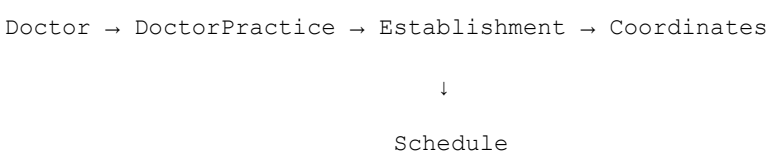
Containers	Docker	Reproducible local/production services

3. High-Level Architecture



4. Core Domain Model

The most important relationship in the platform is:



A doctor may practice at multiple locations and have a different schedule at each location.

- Doctor — identity, name, qualifications, specialties, languages, fee, contact and verification information.
- Establishment — hospital, clinic, nursing home, pharmacy/medical store or doctor chamber.
- DoctorPractice — association between a doctor and a specific establishment.
- Schedule — recurring or temporary schedule belonging to a doctor-practice relationship.
- Availability — current/temporary status distinct from the recurring schedule.
- Location — geographic hierarchy plus latitude/longitude.
- HealthTopic — patient-friendly healthcare concepts such as skin problems, child health or dental problems.
- HealthTopicAlias — English/Bengali phrases mapped to canonical health topics.
- HealthTopicSpecialty — maps health topics to relevant specialties.
- Report — community feedback about incorrect, stale or changed healthcare information.
- MyVisit — patient-created reminder, not a confirmed appointment.

5. Location Resolution Strategy

V1 uses a hybrid location model.

5.1 Current Location

```
Device GPS
   ↓
latitude + longitude
   ↓
selected search origin
```

The device operating system provides the coordinates after the user grants permission. The application does not need a paid map API to obtain the raw GPS coordinates.

5.2 Manual Location

If GPS is unavailable or declined, the user can search for a location:

```
Search city, town, village or PIN
          ↓
Location suggestions
          ↓
User selects one
          ↓
Stored location + coordinates
```

The UX should not force users through State → District → City every time. The hierarchy exists in our database, while the primary interface is direct location search.

5.3 Persisted Location

The selected location can be stored locally on the device so the user does not have to select it repeatedly.

6. Geographic Data Model

Healthcare practice locations should store both human-readable address information and coordinates.

```
practice_locations
------------------
id
name
type
address
phone
latitude
longitude
location  -- PostGIS geography(Point, 4326)
created_at
updated_at
```

The database should maintain a geographic index on the PostGIS location column.

7. Distance Calculation

For conceptual understanding, straight-line geographic distance can be calculated using the Haversine formula:

```
Δlat = lat2 - lat1
Δlon = lon2 - lon1

a = sin²(Δlat/2)
    + cos(lat1) × cos(lat2) × sin²(Δlon/2)

c = 2 × atan2(√a, √(1-a))

distance = R × c
where R ≈ 6371 km
```

For production database searches, use PostGIS ST_DWithin() and ST_Distance() rather than loading every record into Node.js and calculating distances there. This allows spatial indexes to reduce the candidate set efficiently.

8. Core Search Algorithm

The search engine is a multi-stage pipeline:

```
USER QUERY
    ↓
1. Location Resolution
    ↓
2. Query Normalization
    ↓
3. Intent Detection
    ↓
4. Healthcare Concept Mapping
    ↓
5. Candidate Retrieval
    ↓
6. Geographic Filtering
    ↓
7. Schedule / Availability Filtering
    ↓
8. Relevance Ranking
    ↓
9. Result Grouping
    ↓
SEARCH RESULTS
```

9. Query Understanding

The system should recognize several search modes:

- Doctor name: "Dr Amit Das"
- Specialty: "dermatologist"
- Problem: "skin problem"
- Establishment: "Maa Medical Store"
- Location: "doctors in Jhargram"
- Combined: "skin doctor in Jhargram"
- Availability: "skin doctor today"
- Date/time: "child doctor tomorrow evening"

10. Healthcare Taxonomy

Search should translate everyday user language into canonical healthcare concepts. This is routing to relevant care categories, not diagnosing the user.

```
health_topics
--------------
id
name
slug
category

health_topic_aliases
--------------------
id
topic_id
language
phrase

health_topic_specialties
------------------------
id
topic_id
specialty_id

skin problem
skin issue
rash
itching
চামড়ার সমস্যা
চুলকানি
      ↓
canonical topic: skin
      ↓
relevant specialty: dermatology
```

The taxonomy should be editable by administrators so new phrases and relationships can be added without changing application code.

11. Bengali + English Search

English and Bengali should resolve to the same internal concepts. For example:

```
"skin problem"       → skin → dermatology
"চামড়ার সমস্যা"     → skin → dermatology
"child doctor"       → child health → pediatrics
"বাচ্চার ডাক্তার"    → child health → pediatrics
```

The UI and healthcare content should be localized, while provider-entered names, addresses and qualifications generally remain as entered.

12. Candidate Retrieval

Example query: "skin doctor near Jhargram today"

```
Query understanding
    ↓
```

```
topic = skin
specialty = dermatology
location = Jhargram
date = today
    ↓
Find dermatology doctor-practice records
    ↓
Find practice locations within search radius
    ↓
Check applicable schedules
    ↓
Check temporary availability/status
    ↓
Return candidate doctors
```

13. Geographic Search

Search should operate from a coordinate origin plus a radius rather than treating a city as an isolated boundary.

```
Search origin:
Gopiballavpur coordinates

Radius:
25 km

          ↓

Practice locations within radius
          ↓
Nearby doctor candidates
```

If the initial radius has no results, the system may expand the radius in controlled stages, for example 5 km → 10 km → 25 km → 50 km. The UI should clearly communicate when results come from an expanded area.

14. Schedule and Availability

Recurring schedule and real-time/temporary availability must be separate.

```
SCHEDULED
TEMPORARILY_UNAVAILABLE
CONFIRMED_AVAILABLE
UNKNOWN
```

In early V1, a schedule generally means the doctor is expected to practice there at that time; it does not guarantee physical presence.

User-facing examples:

- 🟢 Available today
- 🟡 Scheduled today
- ⚪ Availability unknown
- 🔴 Not available today
- Call to confirm

15. Relevance Ranking

The ranking score represents how relevant a result is to the user's request, not the medical quality or superiority of a doctor.

```
relevanceScore =
    textMatch
  + specialtyMatch
  + problemMatch
  + locationScore
  + availabilityScore
  + scheduleMatch
  + freshnessScore
```

Factor	Importance
Exact doctor/name match	Very high
Exact specialty match	High
Problem → specialty match	High
Requested date/availability match	High
Geographic proximity	Medium-high
Schedule freshness	Medium
Text/fuzzy match	Medium

16. Search Example

User input:

```
📍 Jhargram
🔍 skin problem
```

Query processor output:

```
{
  "intent": "doctor_search",
  "topic": "skin",
  "specialties": ["dermatology"],
  "location": "Jhargram",
  "date": null
}
```

The search engine then:

1. Finds dermatology-related doctors.
2. Finds their practice locations around the selected Jhargram origin.
3. Calculates/filter distances using PostGIS.
4. Checks schedules applicable to the requested date.
5. Considers temporary availability.
6. Ranks results by relevance, proximity and freshness.

17. Search Result Example

```
Dr. Amit Das
Dermatologist

Maa Medical Store
```

```
1.8 km

Thursday
5:00 PM – 8:00 PM

✓ Verified recently

[Call] [Directions]
```

The result should make the doctor, specialty, practice location, schedule and data freshness immediately visible.

18. Directions Without a Map API

The platform does not need to implement navigation. Store destination coordinates and open the device's available map application using the destination latitude/longitude. This keeps navigation outside the V1 platform.

19. No Paid Third-Party API Dependency

Core V1 discovery can operate without:

- Google Maps API
- Google Places API
- Mapbox API
- Algolia
- Elasticsearch Cloud
- OpenAI/Gemini API for every search

The system can use device GPS, our own geographic/location dataset, PostgreSQL, PostGIS, PostgreSQL full-text search and pg_trgm.

20. Why Not Use an LLM for Every Search?

Common healthcare queries should be handled by deterministic taxonomy and PostgreSQL search because this is faster, cheaper and more predictable. An NLP/LLM layer can be added later for complex or previously unseen natural-language queries.

```
Known query
   ↓
Taxonomy / PostgreSQL

Unknown or complex query
   ↓
Optional NLP/LLM layer
   ↓
Canonical search filters
   ↓
PostgreSQL + PostGIS
```

21. Initial Data Management

V1 healthcare data is primarily admin-managed.

- Admin creates healthcare establishments.
- Admin records coordinates and contact information.
- Admin creates doctor profiles.
- Admin links doctors to practice locations.
- Admin adds schedules for each doctor-practice relationship.
- Admin verifies or updates information.
- Community reports are reviewed by admins and do not automatically overwrite data.

22. Search Performance Strategy

- Use indexes on doctor names, establishment names, specialty relationships and aliases.
- Use pg_trgm indexes for fuzzy matching of names and establishment names.
- Use PostgreSQL full-text search for larger text fields where appropriate.
- Use PostGIS spatial indexes for geographic filtering.
- Filter candidates geographically and structurally before applying expensive ranking logic.
- Cache common reference data such as specialty/taxonomy metadata.
- Avoid calculating distance for every database row in application code.

23. Recommended V1 Search Flow

```
Patient opens app
       ↓
Use saved location OR ask for GPS/manual location
       ↓
User enters query
       ↓
Normalize English/Bengali phrases
       ↓
Identify search intent
       ↓
Map problem → healthcare topic → specialty
       ↓
Build geographic search origin + radius
       ↓
Retrieve candidate practice locations
       ↓
Join doctor + practice + schedule
       ↓
Apply date/time/availability filters
       ↓
Calculate relevance
       ↓
Return ranked results
       ↓
Doctor profile / Call / Directions / My Visit
```

24. Future Evolution

The architecture intentionally leaves room for later phases:

- Establishment staff accounts and direct schedule updates.
- Doctor profile claiming and doctor-managed availability.
- Appointment booking and slots.
- Real-time availability signals.
- Dedicated search infrastructure if PostgreSQL is no longer sufficient.
- NLP/LLM-assisted natural-language search.
- Additional Indian languages.
- Diagnostics, blood banks, ambulances and other healthcare discovery categories.

25. Final Technical Baseline

```
PATIENT APP
React Native
Expo
TypeScript
TanStack Query
React Hook Form
Zod
i18next
Expo Notifications

ADMIN WEB
React
Vite
TypeScript
TanStack Query
React Hook Form
Zod
Tailwind CSS
shadcn/ui

BACKEND
Node.js
Express.js
TypeScript
REST API
JWT
Zod
OpenAPI

DATABASE
PostgreSQL
PostGIS
pg_trgm
PostgreSQL Full-Text Search
Drizzle ORM

INFRASTRUCTURE
Linux VPS
Nginx
Docker
Git
HTTPS
```

26. Next Technical Deliverable

The next implementation document should define the complete PostgreSQL schema and API contract: all tables, columns, relationships, constraints, indexes, PostGIS fields, search queries, search service interfaces, authentication boundaries, report workflow and example REST endpoints.